

Descompilando

Entendendo o código da máquina

Anthony Carneiro da Silva

5 de janeiro de 2024

Sumário

1	Índice	5
1.1	Sobre o autor	6
1.2	Sobre o livro	6
2	Introdução a linguagem C	7
2.1	O que é a linguagem C	8
2.1.1	História e contexto	8
2.1.2	Benefícios da linguagem e aplicações práticas	8
2.2	Ambiente de desenvolvimento	9
2.2.1	Configuração do ambiente	9
2.2.2	Primeiro programa	11
2.3	Conceitos Fundamentais	13
2.3.1	Operadores	13
2.3.2	Variáveis e tipos de dados	16
2.3.3	Conversão de tipos	16
2.3.4	Loops e condicionais	16
2.3.5	Funções e bibliotecas	16
2.3.6	Escopo	16
2.3.7	Entrada e saída de dados	16
3	Estrutura de dados e algoritmos em C	17
3.1	Ponteiros	18
3.1.1	Como a memória funciona	18
3.1.2	Endereço de memória	18
3.1.3	Alocação de memória	18
3.1.4	Heap	18
3.1.5	Stack	18
3.2	Strings	18
3.2.1	Por baixo dos panos	18
3.2.2	Declaração	18
3.2.3	Manipulação	18
4	Tópicos avançados	19
4.1	Manipulando arquivos	20
4.1.1	Leitura e escrita de arquivos	20

4.2	Manipulação de bits	20
4.2.1	NOT	20
4.2.2	AND	20
4.2.3	OR	20
4.2.4	XOR	20
5	Depuração e testes	21
5.1	Depuração	22
5.1.1	Estratégias de depuração de código	22
5.1.2	GDB	22
5.1.3	Valgrind	22
5.2	Testes	22
5.2.1	Implementando testes	22
6	Considerações finais	23
6.0.1	Dicas finais	24
6.0.2	Recursos e referências adicionais	24
6.0.3	Mudanças no padrão C2X	24
6.0.4	Obrigado	24

Capítulo 1

Índice

1.1 Sobre o autor

Meu nome é Anthony Carneiro da Silva, tenho 20 anos e sou de Sobral/CE. Desde cedo, sempre gostei de usar o computador para jogar videogame e acessar a internet, ainda cedo tive contato com o mundo da programação, mas assim como muitos na época acreditavam, sempre achei que programação ou desenvolvimento de software fosse algo que só gênios conseguiam fazer.

Aos 17 anos enquanto estava preso a um trabalho que eu já não me sentia mais interessado em fazer, e que sinceramente estava me desgastando, acabei por novamente dar de cara com o mundo da programação ao ver uma notícia de que Harvard estava oferecendo um curso completamente online e gratuito chamado 'CS50', vendo aquilo acabei por dar uma oportunidade a mim mesmo e experimentar.

Por fim, depois de quase 3 anos, posso dizer que começar a estudar programação foi provavelmente uma das melhores escolhas que fiz na vida, além de claro, ter sido extremamente divertido e desafiador. Hoje em dia, não só tenho certeza que quero trabalhar na área mas também que desejo ser plenamente capaz de repassar tudo que aprendi para o mundo. Portanto, espero que esse e-book possa ser o meu primeiro grande passo para divulgar o meu conhecimento, que apesar de não muito, pode ajudar quem está começando.

1.2 Sobre o livro

O intuito desse e-book é ajudar iniciantes na linguagem C, que se sentem perdidos e não sabem por onde começar. Quando comecei a estudar sobre programação percebi de cara que maior parte do conteúdo bom estava disponível apenas em inglês, o que não foi um problema para mim que tive uma educação que me auxiliou e permitiu absorver esse tipo de conteúdo, porém, reconheço que nem todos possuíram a mesma oportunidade de aprender inglês ao decorrer da vida.

Aqui você será guiado desde a como preparar a sua máquina para começar a programar, a até algumas dicas e truques mais avançados sobre como a linguagem e o compilador se comportam. Portanto, espero que esse material seja suficiente para preencher todas as lacunas de conhecimento necessárias para poder começar a programar em C sem muitos problemas.

Capítulo 2

Introdução a linguagem C

2.1 O que é a linguagem C

2.1.1 História e contexto

A linguagem C foi criada em 1970 por Dennis Ritchie com a finalidade de desenvolver sistemas operacionais, drivers e outros softwares de baixo nível. Apesar de ter mais de 60 anos, continua sendo amplamente utilizada em diversos tipos de hardware, desde supercomputadores até microcontroladores e sistemas embarcados.

Com compiladores disponíveis em praticamente todos os sistemas modernos, destacou-se pela portabilidade e foi amplamente empregada na construção de utilitários e aplicações complexas que exigiam maior velocidade e eficiência. Em 1978, foi publicado o livro "The C Programming Language", escrito pelo coautor da linguagem C, que por muito tempo serviu como referência padrão para a linguagem.

Por ser uma linguagem imperativa, com suporte à programação estruturada, amplo escopo lexical e acesso eficiente à memória do computador em baixo nível, sendo frequentemente a escolha ideal para projetos em sistemas com grandes restrições de hardware.

2.1.2 Benefícios da linguagem e aplicações práticas

Devido ao seu alto desempenho como linguagem compilada de baixo nível, a programação em C possui vantagens e desvantagens. Entre as vantagens, destacam-se:

- Alta portabilidade
- Alto desempenho
- Gerenciamento eficiente de memória
- Acesso a recursos do sistema

No entanto, essas vantagens têm um preço, ideia essa que pode ser entendida melhor por uma frase que gosto bastante (a mesma encontrada em inglês no verso do livro).

"No começo todos queremos resultados, mas no fim, tudo que queremos é controle"¹

Essa frase se mostra verdadeira, pois apesar de ter total controle e poder sobre as funcionalidades da linguagem, quando mal utilizadas elas podem criar vulnerabilidades de segurança e uma complexidade excessiva, que tende a aumentar ao longo de um projeto.

Apesar dos desafios, quando dominada e compreendida adequadamente, a linguagem C não apenas oferece um entendimento sólido dos fundamentos da programação, mas também encontra aplicação prática no desenvolvimento de:

¹Eskil Steenberg

- Sistemas operacionais
- Microcontroladores
- Drivers
- Linguagens de programação
- Utilitários

2.2 Ambiente de desenvolvimento

2.2.1 Configuração do ambiente

Configurar o ambiente de desenvolvimento para programar em C é crucial para iniciar projetos com eficiência. Abaixo, são fornecidos passos para configurar o ambiente em sistemas Linux e Windows.

Linux

Para configurar o ambiente de desenvolvimento C em sistemas Linux, siga estes passos:

1. **Instalação do compilador C (GCC):** Em maior parte das distribuições Linux o GCC já está disponível por padrão, mas caso não esteja, ele pode ser facilmente instalado utilizando o gerenciador de pacotes da sua distribuição (como apt para Ubuntu/Debian ou yum para Fedora).

Segue abaixo uma lista de comandos para instalar o GCC nas distribuições Linux mais famosas:

```
# Debian
sudo apt update
sudo apt install build-essential
# Red Hat-based (Fedora, CentOS)
sudo dnf update
sudo dnf install gcc
# Arch Linux
sudo pacman -Syu
sudo pacman -S gcc
# openSUSE
sudo zypper refresh
sudo zypper install gcc
# Alpine Linux
sudo apk update
sudo apk add build-base
```

```
# Para checar se o GCC foi instalado
# corretamente use o comando abaixo
gcc --version
```

Windows

Para configurar o ambiente de desenvolvimento C em sistemas Windows, siga estes passos:

1. **Instalação do compilador C (GCC):** No Windows a utilização de ferramentas como o GCC acaba por ser um pouco mais complexa pois é necessária a instalação do MinGW ou MSYS2. Essas ferramentas fornecem um ambiente similar ao Linux para compilar código C.

Para instalar o MinGW, siga os passos abaixo:

- (a) Acesse o site oficial do MinGW: <https://osdn.net/projects/mingw/>
- (b) Baixe o instalador adequado para o seu sistema (32 ou 64 bits).
- (c) Execute o instalador baixado (`mingw-w64-install.exe`).
- (d) Durante a instalação, selecione os pacotes desejados. O pacote essencial para o GCC é o `mingw32-gcc` para sistemas 32 bits ou `mingw64-gcc` para sistemas 64 bits.
- (e) Siga as instruções do instalador para concluir a instalação.

Após a instalação, é necessário configurar o ambiente para usar o GCC via linha de comando:

- (a) Adicione o diretório bin do MinGW ao caminho do sistema:
 - Para Windows 10, pesquise "Editar variáveis de ambiente do sistema" na barra de pesquisa e clique em "Variáveis de ambiente". Em "Variáveis do sistema", selecione a variável PATH e clique em "Editar...". Adicione o caminho para o diretório bin do MinGW (exemplo: `C:\MinGW\bin`) e clique em OK.
 - Para versões anteriores do Windows, o processo é similar, mas pode variar um pouco, caso você utilize uma versão mais antiga do sistema talvez seja necessário que você faça uma pesquisa a parte para finalizar o processo corretamente.
- (b) Verifique se a instalação foi bem-sucedida abra o Prompt de Comando (`cmd`) e insira o seguinte comando:

```
gcc --version
```

2. **Editor de texto ou IDE:** Instale um editor de texto ou uma IDE (Ambiente de Desenvolvimento Integrado), para escrever e compilar código C. Alguns exemplos populares são o Visual Studio Code, Atom, Sublime Text, ou IDEs como o Code::Blocks ou o Eclipse com plugins para C/C++.

Eu pessoalmente recomendo a utilização do Neovim que apesar de ser uma ferramenta de difícil utilização, pode acabar por aumentar muito a sua produtividade no médio/longo prazo.

Se você seguiu os passos acima com sucesso, seu ambiente de desenvolvimento C está configurado e pronto para uso. Agora é o momento de escrever seu primeiro programa!

2.2.2 Primeiro programa

Após configurar e instalar todas as ferramentas necessárias, é comum começar com um simples “Olá, mundo!”.

O programa que iremos fazer exibe o texto “Olá, mundo!” na tela e embora seja simples, é um ponto de partida essencial para começar a entender não somente como a linguagem C funciona, mas qualquer outra linguagem de programação.

Para criar o seu primeiro programa, siga estes passos:

1. Crie um diretório para guardar o seu projeto.
2. Crie um arquivo chamado “main.c” onde será escrito o código.
3. Abra o arquivo “main.c” no seu editor de código e escreva o seguinte:

```
#include <stdio.h>

int main(void) {
    printf("Olá, mundo!\n");
    return 0;
}
```

4. Salve o arquivo de texto e compile o código com o GCC usando o seguinte comando no seu terminal:

```
gcc main.c
```

5. Agora, execute o programa compilado com o seguinte comando:

```
# Para sistemas Linux
./a.out
# Para sistemas Windows
./a.exe
```

Após executar o comando acima, você já deve ver o texto “Olá, mundo!” na sua tela. Mas antes de prosseguir, você provavelmente está se perguntando o que exatamente cada uma das linhas de código que você escreveu fazem e como exatamente elas fazem esse texto aparecer na sua tela. Por isso, vou explicar passo a passo o que significa cada linha do código acima.

```
#include <stdio.h>
```

Na linha número 1, temos o “include”, uma diretiva que inclui bibliotecas externas no nosso programa. Aqui, estamos incluindo a biblioteca “stdio.h”. O conceito de bibliotecas ainda vai ser melhor explicado futuramente, mas você pode entender como se cada biblioteca fosse um tipo de extensão com funções extras.

```
int main(void) {
```

Na linha número 2, temos o início de uma função. Nessa linha, há o uso de quatro palavras/caracteres importantes:

- **”int”**: Define o que a função deve retornar ao ser finalizada, neste caso, um número inteiro.
- **”main”**: É o nome que foi escolhido para o bloco de código principal. Apesar de não ser necessariamente obrigatório usar o nome **main** para a sua função principal, acabar por ser uma boa prática a ser seguida.
- **”(void)”**: Define quais parâmetros/argumentos são necessários para chamar a função. Neste caso, a palavra **void** indica que a função não precisa de nenhum valor para ser chamada.
- **”{”**: Marca o início de um bloco de código de uma função.

```
printf("Olá, mundo!\n");
```

Na linha número 3, encontramos a diretiva “printf”, responsável por exibir informações no terminal. Esta diretiva na verdade não existe como uma funcionalidade padrão na linguagem C, e está sendo implementada através da biblioteca “stdio.h”, mencionada na linha 1.

Neste caso a informação que está sendo passada para a função é o texto “Olá, mundo!\n” que sempre deve ser colocado entre os parênteses e as aspas, você pode ter percebido também a presença do caracter “\n” que não aparece quando a mensagem é exibida. Esse caracter é um caracter especial e tem como função fazer com que naquele espaço da mensagem o terminal desça uma linha abaixo antes de continuar.

```
return 0;
```

Na linha número 4, encontramos o comando “return”. Em uma função, essa diretiva indica o valor que a função deve retornar quando terminar sua execução e normalmente, mas nem sempre, é acompanhada de um valor que está escrito como um sufixo da diretiva.

No caso da função “main”, que é a função principal em um programa C, o valor 0 geralmente é usado como uma boa prática para indicar que o programa foi finalizado sem erros.

```
}
```

A linha número 5 é a chave de fechamento “}”, que indica o fim do bloco de código da função “main”. Tudo o que está contido entre as chaves forma o corpo da função, o compilador (GCC) utiliza dessas chaves de abertura e fechamento como uma forma de entender quando um código é pertencente a uma função.

Caso tenha parecido um pouco confuso a primeira vista, acredito que com um pouco de prática você vai começar a escrever esse tipo de código sem sequer pensar direito sobre isso. E no geral, essa estrutura tende a se repetir bastante, então caso não tenha entendido algo muito bem vale a pena voltar e reler.

Exercícios

É de extrema importância que você realize os exercícios aqui elaborados para fixar o conteúdo.

1. Modifique a mensagem exibida na tela pela função “printf”.
2. Tente alterar a posição do caracter especial “\n” e veja o que acontece.
3. Mudar o nome da função para qualquer outra coisa e veja qual o erro retornado.
4. Remova a linha de código que inclui a biblioteca “stdio.h” e veja qual o erro retornado.

2.3 Conceitos Fundamentais

2.3.1 Operadores

Um operador é um símbolo que indica a realização de uma operação sobre um, ou mais, valores de um programa. Na linguagem C é possível encontrar alguns diferentes tipos de operadores, dentre eles:

Aritméticos

Os operadores aritméticos são utilizados para representar operações matemáticas entre valores. Segue abaixo uma lista de todos os operadores aritméticos:

Operador	Descrição	Exemplo	Resultado
+	soma	$10 + 20$	30
-	subtração	$20 - 10$	10
*	multiplicação	$10 * 2$	20
/	divisão	$20 / 10$	2
%	módulo	$20 \% 11$	1
++	incremento	++10 é o mesmo que $10 + 1$	11
--	decremento	--10 é o mesmo que $10 - 1$	9

Relacionais

Os operadores relacionais possuem a finalidade de fazer comparações entre valores, como por exemplo, verificar se um número é maior que outro. O resultado de uma comparação sempre vai ser **verdadeiro** ou **falso**.

Segue abaixo uma lista dos comparadores relacionais:

Operador	Descrição	Exemplo	Resultado
==	igual	$10 == 10$	verdadeiro
!=	diferente	$10 != 10$	falso
>	maior que	$10 > 10$	falso
>=	maior ou igual	$10 >= 10$	verdadeiro
<	menor que	$10 < 10$	falso
<=	menor ou igual	$10 <= 10$	verdadeiro

Lógicos

Os operadores lógicos são usados em conjunto com os operadores relacionais. Para compreendê-los corretamente, é importante entender que na linguagem C, os valores de **verdadeiro** e **falso** podem ser representados pelos números 1 e 0, respectivamente.

Ao utilizar os operadores lógicos, podemos criar verificações de valores mais complexas. Abaixo está a lista dos operadores existentes e como usá-los:

Operador	Descrição	Exemplo	Resultado
&&	AND lógico	1 && 1	1
	OR lógico	1 0	1
!	NOT lógico	!1	0

Observação:

Caso tenha ficado um pouco confuso como cada operador atua exatamente, vou tentar detalhar de maneira mais sucinta e com mais exemplos de como cada operador deve agir em cada caso.

1. **AND lógico:** Retorna **verdadeiro** no caso de ambos os valores serem verdadeiros, caso contrário, retorna **falso**.

Exemplo	Resultado
0 && 0	0
0 && 1	0
1 && 0	0
1 && 1	1

2. **OR lógico:** Retorna **verdadeiro** se pelo menos um dos valores for verdadeiro, caso contrário, retorna **falso**.

Exemplo	Resultado
0 0	0
0 1	1
1 0	1
1 1	1

3. **NOT lógico:** Inverte o valor lógico, isto é, se for verdadeiro, torna-se falso, e se for falso, torna-se verdadeiro.

Exemplo	Resultado
!0	1
!1	0

Atribuição

Os operadores de atribuição tem a função de atribuir valores a variáveis. Por exemplo, utilizando um operador de atribuição poderíamos dizer que $A = 10$ então toda vez que utilizássemos o valor A no código, o programa entenderia que estaríamos nos referindo ao número 10.

Operador	Descrição	Exemplo	Equivale
=	Atribuição de valor a variável	$A = 10$	
+=	Atribuição com soma	$A += 10$	$A = A + 10$
-=	Atribuição com subtração	$A -= 10$	$A = A - 10$
*=	Atribuição com multiplicação	$A *= 10$	$A = A * 10$
/=	Atribuição com divisão	$A /= 10$	$A = A / 10$
%=	Atribuição com divisão	$A \% = 10$	$A = A \% 10$

2.3.2 Variáveis e tipos de dados

2.3.3 Conversão de tipos

2.3.4 Loops e condicionais

2.3.5 Funções e bibliotecas

2.3.6 Escopo

2.3.7 Entrada e saída de dados

Capítulo 3

Estrutura de dados e algoritmos em C

3.1 Ponteiros

3.1.1 Como a memória funciona

3.1.2 Endereço de memória

3.1.3 Alocação de memória

3.1.4 Heap

3.1.5 Stack

3.2 Strings

3.2.1 Por baixo dos panos

3.2.2 Declaração

3.2.3 Manipulação

Capítulo 4

Tópicos avançados

4.1 Manipulando arquivos

4.1.1 Leitura e escrita de arquivos

4.2 Manipulação de bits

4.2.1 NOT

4.2.2 AND

4.2.3 OR

4.2.4 XOR

Capítulo 5

Depuração e testes

5.1 Depuração

5.1.1 Estratégias de depuração de código

5.1.2 GDB

5.1.3 Valgrind

5.2 Testes

5.2.1 Implementando testes

Capítulo 6

Considerações finais

6.0.1 Dicas finais

6.0.2 Recursos e referências adicionais

6.0.3 Mudanças no padrão C2X

6.0.4 Obrigado